

LAB use case UDF, write in the proper format , whichever we are change from system to system tell that me "<<>>" write that inside it so we know that when what we have to change as per changes in use cases or system get change.

Here is the complete step-by-step procedure for **Hive Custom Java UDF (Use Case 5: Mobile App Activity Log Analytics)** organized into two groups based on system boundaries and execution context.

TXT

All variable or system-dependent values (such as file paths, directories, table names, custom class names, JAR names, hostnames, and ports) that need to be modified when systems or use cases change are enclosed inside `<< >>`.

## Group A: System & Storage Operations (Linux / HDFS / Java Compilation)

### Terminal 1: HDFS Directory Setup, Local File Preparation, & Java UDF Compilation

#### 1. Verify Hadoop Services and File System:

Bash



```
start-dfs.sh
start-yarn.sh
jps
hdfs dfs -ls /
```

#### 1. Create Local Working Directory Structure:

Bash



```
mkdir -p ~/<<hive-app-logs>>/<<data>>[cite: 1]
mkdir -p ~/<<hive-app-logs>>/<<udf>>[cite: 1]
```

```
cd ~/<<hive-app-logs>>[cite: 1]
```

## 1. Create Local CSV Log File:

Bash



```
nano <<data/app_activity.csv>>
```

*File Content:*

Plaintext



```
<<user_id>>,<<user_name>>,<<event_type>>,<<device_type>>,<<event_date>>,<<session_dur  
<<101>>,<<rahul>>,<<LOGIN>>,<<Android>>,<<2026-10-01>>,<<120>>[cite: 1]  
<<102>>,<<priya>>,<<PURCHASE>>,<<iOS>>,<<2026-10-01>>,<<300>>[cite: 1]  
<<103>>,<<amit>>,<<LOGOUT>>,<<Android>>,<<2026-10-02>>,<<45>>[cite: 1]
```

## 1. Upload Log Data to HDFS Raw Directory:

Bash



```
hdfs dfs -mkdir -p /<<mobile-app>>/<<raw>>[cite: 1]  
hdfs dfs -put -f <<data/app_activity.csv>> /<<mobile-app>>/<<raw>>/[cite: 1]  
hdfs dfs -ls /<<mobile-app>>/<<raw>>[cite: 1]  
hdfs dfs -cat /<<mobile-app>>/<<raw>>/<<app_activity.csv>>[cite: 1]
```

## 1. Write the Java UDF Code:

Bash



```
cd ~/<<hive-app-logs>>/<<udf>>[cite: 1]  
nano <<UpperCaseUDF.java>>[cite: 1]
```

## File Content:

Java



```
import org.apache.hadoop.hive.ql.exec.UDF;[cite: 1]
import org.apache.hadoop.io.Text;[cite: 1]

public class <<UpperCaseUDF>> extends UDF {[cite: 1]
    public Text evaluate(Text input) {[cite: 1]
        if (input == null) {[cite: 1]
            return null;[cite: 1]
        }
        return new Text(input.toString().toUpperCase());[cite: 1]
    }
}
```

### 1. Compile Java UDF and Package into JAR:

Bash



```
export HIVE_HOME=/<<opt>>/<<hive>>[cite: 1]
echo $HIVE_HOME[cite: 1]

# Compile Java code against Hadoop & Hive Classpath
javac -cp "$HIVE_HOME/lib/*:/<<opt>>/<<hadoop>>/share/hadoop/common/*" <<UpperCaseUDF

# Verify compilation class file
ls -lh <<UpperCaseUDF.class>>[cite: 1]

# Create JAR File
jar -cf <<uppercase-udf.jar>> <<UpperCaseUDF.class>>[cite: 1]
ls -l <<uppercase-udf.jar>>[cite: 1]
```

## Group B: Hive Warehouse & UDF Execution

### Terminal 2: Hive CLI / Beeline Execution

#### 1. Connect to Hive Service via Beeline:

Bash



```
beeline -u jdbc:hive2://<<localhost>>:<<10000>>/<<default>>[cite: 1, 5]
```

## 1. Create Hive Database & Set Context:

SQL



```
CREATE DATABASE IF NOT EXISTS <<mobile_app>>;[cite: 1]  
USE <<mobile_app>>;[cite: 1]
```

## 1. Create External Table Over HDFS Raw Logs:

SQL



```
CREATE EXTERNAL TABLE <<app_activity>> (  
    <<user_id>> INT,  
    <<user_name>> STRING,  
    <<event_type>> STRING,  
    <<device_type>> STRING,  
    <<event_date>> DATE,  
    <<session_duration>> INT  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY '<<, >>'  
LOCATION '/<<mobile-app>>/<<raw>>';[cite: 1, 5]  
  
SELECT * FROM <<app_activity>>;[cite: 1]
```

## 1. Register Custom UDF Jar & Create Temporary/Permanent Function:

SQL



```
-- Add JAR to Hive Classpath  
ADD JAR /<<home>>/<<ubuntu>>/<<hive-app-logs>>/<<udf>>/<<uppercase-udf.jar>>;[cite: 1]
```

```
-- Register Custom UDF Function
CREATE FUNCTION <<uppercase_name>> AS '<<UpperCaseUDF>>';[cite: 1]
```

## 1. Test UDF and Run Analytics Queries:

SQL



```
-- Test UDF Output
SELECT <<user_name>>, <<uppercase_name>>(<<user_name>>) AS <<transformed_name>> FROM

-- Analyze Event Types
SELECT <<event_type>>, COUNT(DISTINCT <<user_id>>) AS <<total_users>>
FROM <<app_activity>>
GROUP BY <<event_type>>;[cite: 1]

-- Calculate Average Session Duration per Device
SELECT <<device_type>>, AVG(<<session_duration>>) AS <<avg_duration>>
FROM <<app_activity>>
GROUP BY <<device_type>>;[cite: 1]
```